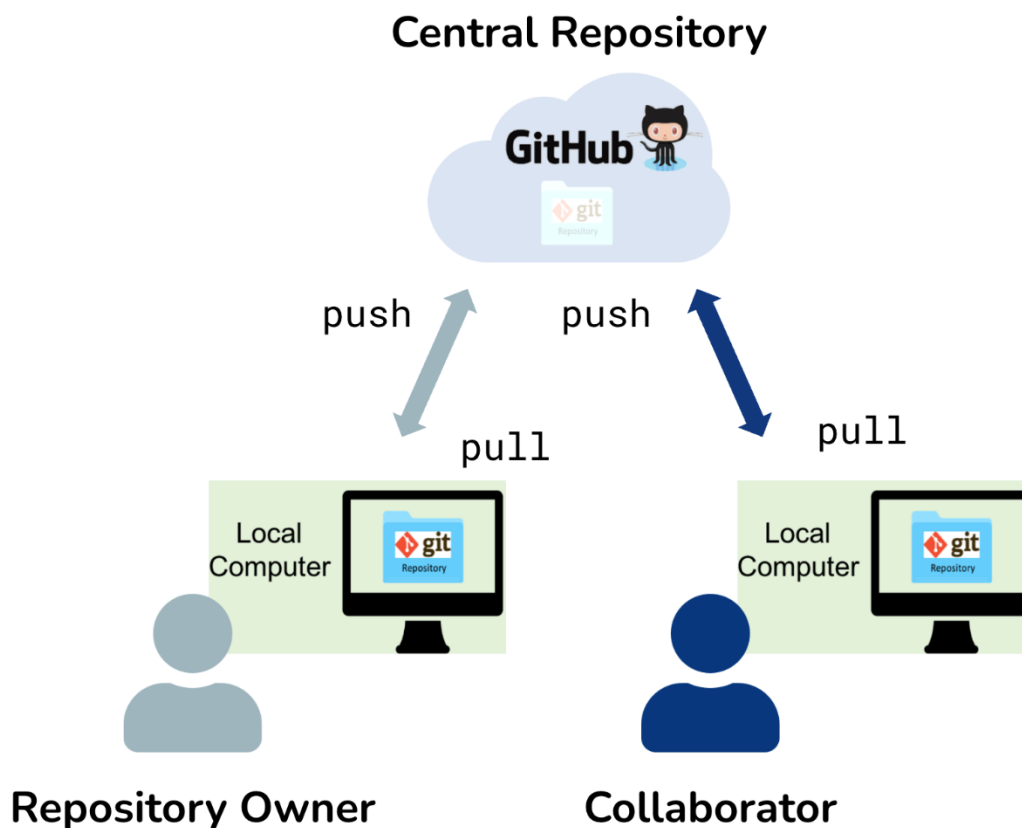


Industrial Tool

Git & Github

What is Collaboration in Git?

- Collaboration allows multiple people to work on the same project together.
- It is similar to creating a group on GitHub like groups in social media.
- Collaborators added to the project can push and merge changes.
- They can also perform other actions to help manage the project.



Types of Collaboration in Git :-

1. Solo Collaboration

- A single developer works alone on a project.
- Git helps manage their own changes and version history.
- Branches allow them to work independently without affecting the main code.

2. Team Collaboration

- Multiple developers work together on one codebase.

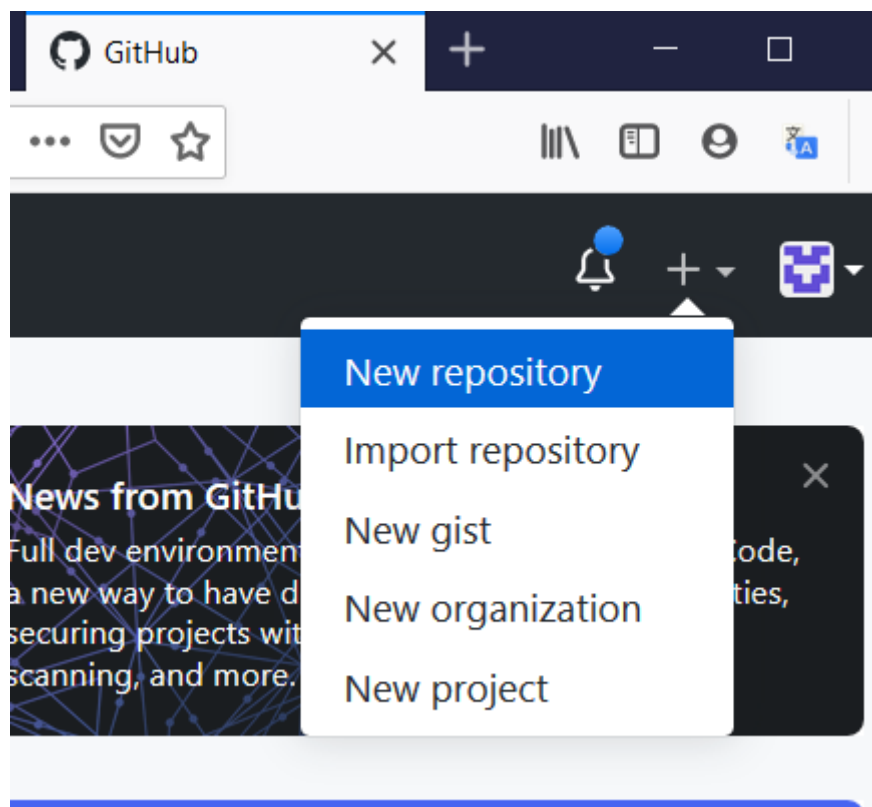
- Git provides branching, merging, and pull requests for coordination.
- Developers work separately and later merge and review each other's changes.

3. Open Source Collaboration

- Developers contribute to public Git projects.
- They fork the repository, make changes, and send pull requests.
- Maintainers review and merge contributions that meet the project standards.

Steps for Collaboration in GIT :-

Step 1: Create a Repository



Create Git Repository

Step 2: Add files to your project.

Open Git Bash into your local working directory where files are saved and follow the commands:

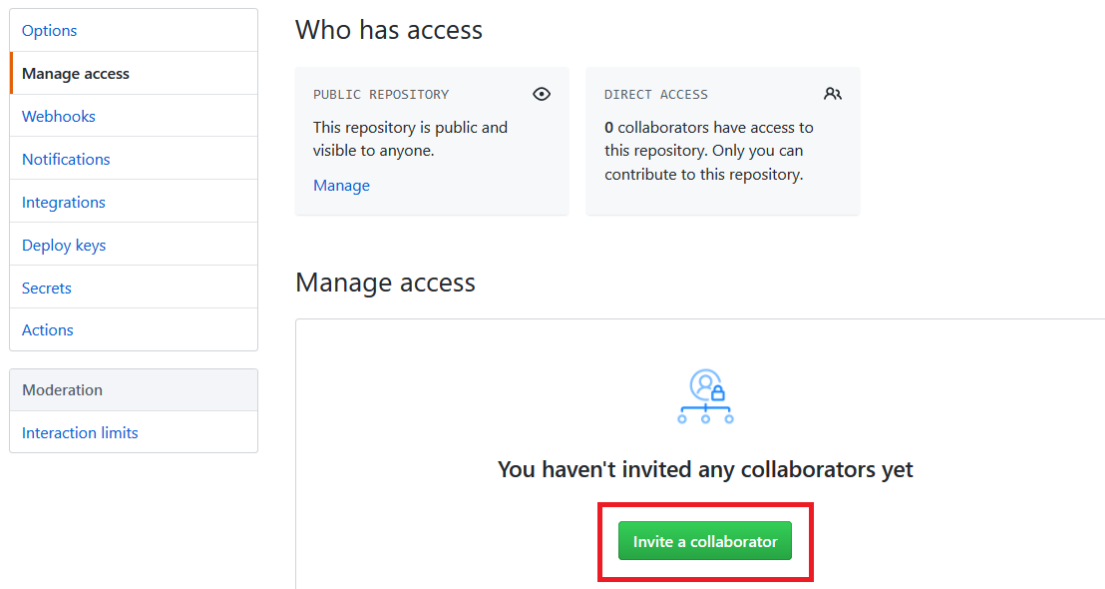
```
$ git init  
$ git add .  
$ git commit -m "initial commit"
```

```
$ git remote add origin remote repository URL
```

```
$ git remote -v
```

```
$ git push -f origin master
```

Step 3: Adding your Collaborators. Click on settings and then follow the below steps :



After clicking on "Invite a collaborator" fill in the required details and then you are done.

Note: In order to perfectly work in a collaborative team you need to know below terminologies and their applications.

Key Git Features for Collaboration :-

- 1. Branching:** Developers can create branches for new features, fixes, or experiments.
This lets them work separately without affecting the main code.
- 2. Merging:** Changes from a branch can be merged back into the main branch.
Git helps handle this and resolves many conflicts automatically.
- 3. Version Control:** Git records every change in the project's history.
Developers can revert changes or review who made updates and why.
- 4. Distributed System:** Every developer has a complete copy of the repository.
This allows offline work and full access to the project history.
- 5. Pull Requests:** Platforms like GitHub let developers review code before merging.
This ensures better quality and teamwork in the project.

Benefits of Collaboration in Git :-

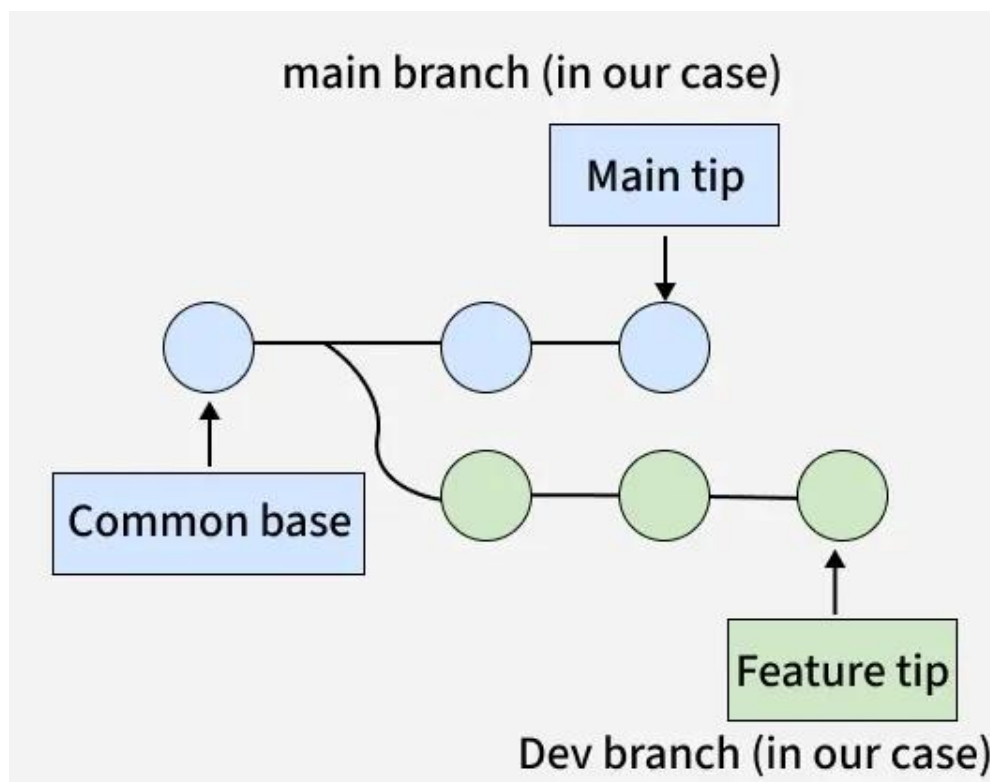
- 1. Improved Code Quality:** Code reviews help find bugs and improve the overall code.
- 2. Efficiency:** Teams can work on different tasks at the same time without interference.
- 3. Version History:** Git stores all changes, making it easy to track issues and restore older versions.
- 4. Reduced Conflicts:** Branching and pull requests help avoid and resolve conflicts smoothly.

Git – Merge :-

- Git Merge is used to combine changes from two branches into one.
- It brings together work from different branches into a single history.
- No progress is lost during the merge.
- Example: merging a feature branch into the main branch to add new updates.

Working of Git Merge :-

Before Merge -



This image shows the state of the repository before the merge takes place.

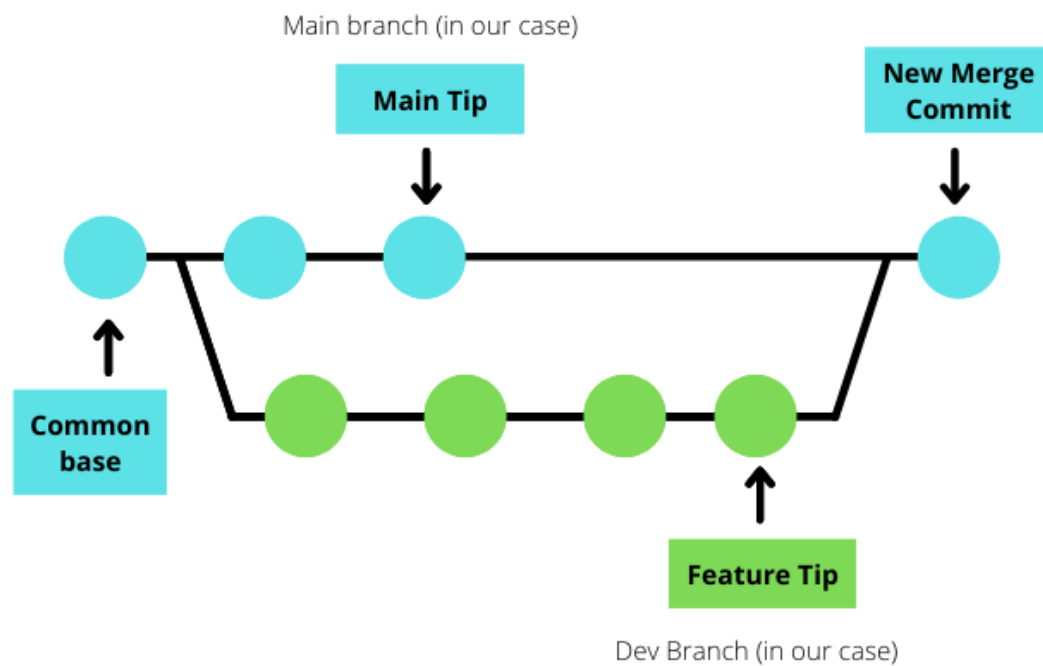
Key Elements

1. Common Base: The commit from which both the main and feature (dev) branches originated.
2. Main Branch (in blue): This is your production or main branch where stable code lives.
3. Feature/Dev Branch (in green): This branch is where new development work has occurred.
4. Main Tip: The latest commit on the main branch.
5. Feature Tip: The latest commit on the feature branch.

Interpretation

- Development has happened in parallel on both the main and feature branches.
- These branches diverged from a common ancestor (the Common Base).
- You are now preparing to merge the feature branch back into the main branch.

After Merge –



This image shows what happens after executing a git merge.

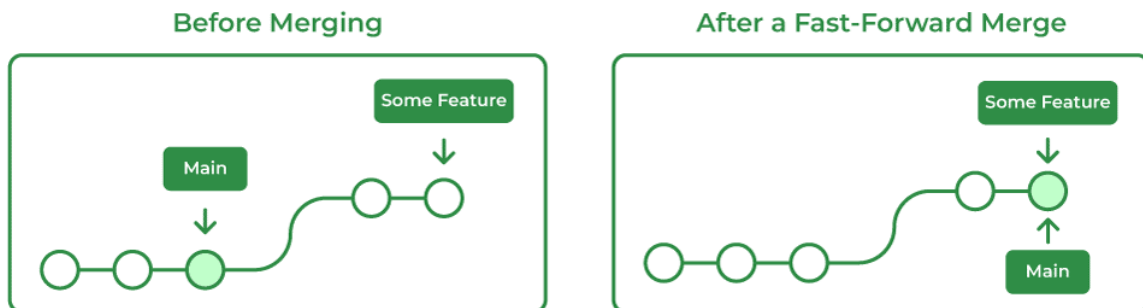
- During a merge, Git compares three commits: Main Tip, Feature Tip, and Common Base.
- If there are no conflicts, Git automatically merges both branches.
- It creates a new merge commit that combines the histories.

Types of Merging in Git :-

Git supports several types of merging. The two most common types are:

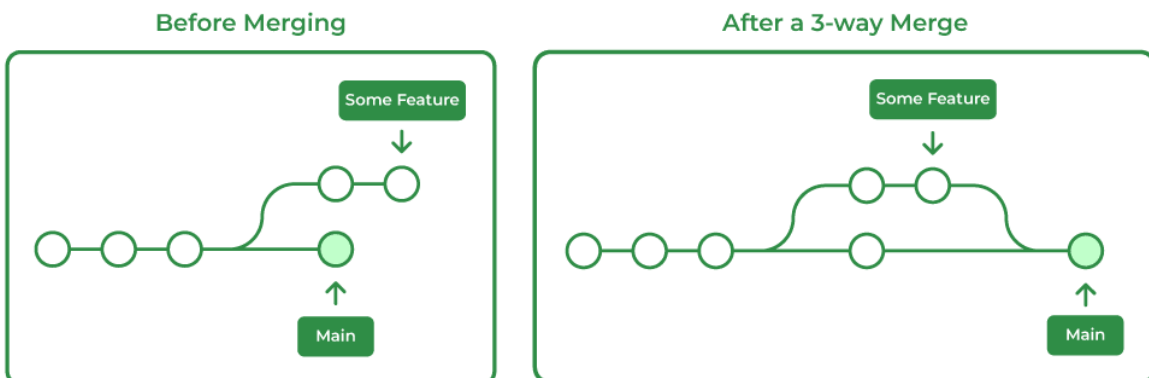
1. Fast-forward Merging

- Happens when the main branch is directly ahead of the feature branch.
- Git just moves the branch pointer forward without creating a merge commit.
- This works only when the branches have not diverged.



2. Three-way Merging

- Used when both branches have new changes since they split.
- Git compares the main branch, feature branch, and their common base.
- A new merge commit is created to combine all changes.



Steps to Perform Git Merge :-

To ensure smooth merging, follow these steps:

Step 1: Create a New Branch

Create a new branch from the remote repository you want to merge.

```
git branch <new-branch-name>
```

Step 2: Pull the Latest Changes

Before merging, ensure that you pull the latest changes from both branches (e.g., main and the feature branch).

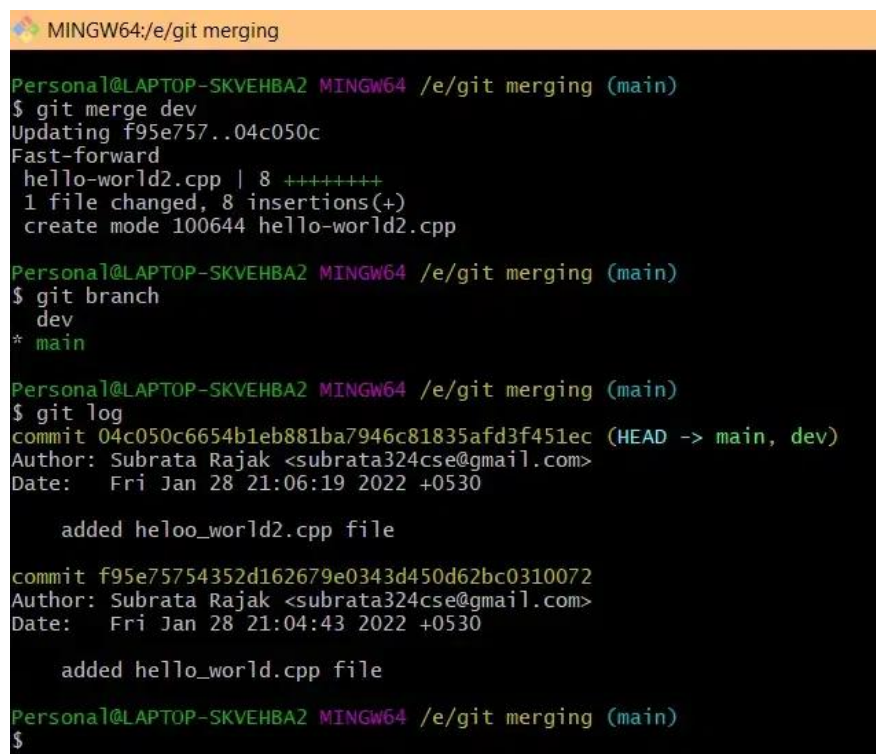
```
git checkout <target-branch>
git pull origin <target-branch>
```

```
git checkout <feature-branch>
git pull origin <feature-branch>
```

Step 3: Merge the Branch

If any conflicts arise, Git will notify you. Resolve them manually before proceeding.

```
git checkout <target-branch>
git merge <feature-branch>
```

A terminal window titled 'MINGW64:/e/git merging' showing a series of Git commands and their outputs. The user is in the 'main' branch. They run 'git merge dev', which updates the main branch with changes from 'dev' (commit f95e757..04c050c) in a fast-forward manner, adding 8 lines to 'hello-world2.cpp'. Then they run 'git branch', showing 'dev' and 'main' branches. Next, they run 'git log', showing the commit history for both branches. The log shows two commits: one for 'dev' (04c050c) adding 'heloo_world2.cpp' and one for 'main' (f95e757) adding 'hello_world.cpp'.

```
MINGW64:/e/git merging

Personal@LAPTOP-SKVEHBA2 MINGW64 /e/git merging (main)
$ git merge dev
Updating f95e757..04c050c
Fast-forward
 hello-world2.cpp | 8 +++++++
 1 file changed, 8 insertions(+)
 create mode 100644 hello-world2.cpp

Personal@LAPTOP-SKVEHBA2 MINGW64 /e/git merging (main)
$ git branch
  dev
* main

Personal@LAPTOP-SKVEHBA2 MINGW64 /e/git merging (main)
$ git log
commit 04c050c6654b1eb881ba7946c81835afd3f451ec (HEAD -> main, dev)
Author: Subrata Rajak <subrata324cse@gmail.com>
Date:   Fri Jan 28 21:06:19 2022 +0530

    added heloo_world2.cpp file

commit f95e75754352d162679e0343d450d62bc0310072
Author: Subrata Rajak <subrata324cse@gmail.com>
Date:   Fri Jan 28 21:04:43 2022 +0530

    added hello_world.cpp file

Personal@LAPTOP-SKVEHBA2 MINGW64 /e/git merging (main)
$
```

Step 4: Test the Merged Code

Make sure the merged code functions correctly by testing it either automatically or manually.

Run tests or manually test your application

Step 5: Commit the Merged Code

Once satisfied with the merged code, commit the changes:

```
git commit -m "Merge branch 'dev' into main"
```

Step 6: Push the Merged Branch

Push the changes to the remote repository to share the new merged branch:

```
git push origin main
```

Git Clone :-

- git clone creates a copy of an existing Git repository.
- The repository can be on GitHub, GitLab, Bitbucket, or any server.
- It downloads the complete project, including history and all branches.

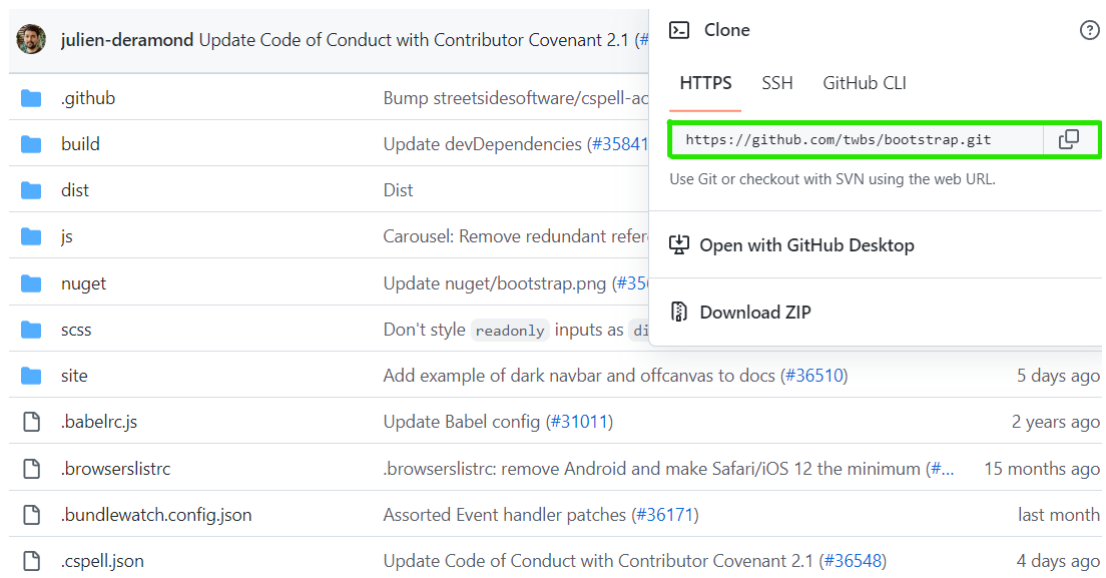
Syntax:

git clone <repository-link>

For Example:

If you want to clone the Bootstrap repository, you would run.

git clone https://github.com/twbs/bootstrap



What is Git Clone?

To clone a repository, copy its link, go to your desired folder using cd, and run git clone <repo-link> to download it locally.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~ (master)
$ cd Desktop/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop (master)
$ mkdir Git

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop (master)
$ cd Git/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$
```

What is Git Clone?

After that, Clone the repository using git clone <repo-link>

git clone https://github.com/twbs/bootstrap

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git clone https://github.com/twbs/bootstrap.git
Cloning into 'bootstrap'...
remote: Enumerating objects: 181364, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (13/13), done.
Receiving objects: 14% (25606/181364), 16.08 MiB | 839.00 KiB/s
```

What is Git Clone?

The cloning process will begin. You'll see progress messages indicating how much of the repository has been cloned (e.g., 14%, 50%, 100%).

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git clone https://github.com/twbs/bootstrap.git
Cloning into 'bootstrap'...
remote: Enumerating objects: 184546, done.
remote: Total 184546 (delta 0), reused 0 (delta 0), pack-reused 184546
Receiving objects: 100% (184546/184546), 209.26 MiB | 1.59 MiB/s, done.
Resolving deltas: 100% (120314/120314), done.
Updating files: 100% (712/712), done.
```

Once the cloning process is complete, you can navigate into the new directory (e.g., cd bootstrap) to start working on the project locally.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ ls
bootstrap/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ cd bootstrap/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git/bootstrap (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git/bootstrap (main)
$ git log
commit c3c65911665ab64bdaa15d405db65ee81655dbf3 (HEAD -> main, origin/main, origin/HEAD)
Author: Kitty Giraudel <1889710+KittyGiraudel@users.noreply.github.com>
commit c3c65911665ab64bdaa15d405db65ee81655dbf3 (HEAD -> main, origin/main, origin/HEAD)
Author: Kitty Giraudel <1889710+KittyGiraudel@users.noreply.github.com>
Date: Tue Aug 9 16:14:51 2022 +0200

    Update an outdated name mentioned in a Sass comment
```

If you need to stop the cloning process before it finishes, you can press Ctrl + C to cancel the operation.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git clone https://github.com/twbs/bootstrap.git
Cloning into 'bootstrap'...
remote: Enumerating objects: 184546, done.
fatal: fetch-pack: invalid index-pack output5 MiB | 2.58 MiB/s
fetch-pack: unexpected disconnect while reading sideband packet
```

By cloning a repository, you get a complete copy of the project, including all of its branches and commit history, allowing you to work offline and push your changes back once you are ready.

Git Clone with Username & Password :-

- Used when cloning via HTTPS and authentication is required.
- Not recommended to include passwords in URLs — use tokens or credential managers instead.

Cloning a Specific Branch :-

- Use -b or --branch option to clone a particular branch:
git clone -b <branch-name> <repo-url>

Repo-to-Repo Collaboration :-

- Cloning lets you work on a local copy and later push changes back for review:
git clone <remote_repository_url>

Git Clone Options :-

1. Mirror Clone

- Makes a full backup with all refs and configs.
git clone --mirror <repo-url>

2. Bare Clone

- Contains only version control data, no working files.
git clone --bare <repo-url>
- Used for central/shared repositories.

Git Fetch :-

What is Git Fetch?

- git fetch gets updates from a remote repository.
- It does not merge those updates into your local branch.
- It downloads new commits, branches, and tags for review.
- You can choose when to apply the changes manually.

Key Points of Git Fetch :-

1. **Non-disruptive:** Does not change local files or the current branch.
2. **Safe for review:** Lets you inspect updates before merging.

How Git Fetch Works :-

- git fetch connects to the remote repository (usually origin).
- It downloads new commits and branches to your local repository.
- Updates are stored in remote-tracking branches (e.g., origin/main).
- Local branches are not changed automatically.
- You can choose when to merge or rebase the fetched changes.

When to Use Git Fetch :-

1. **Collaborative Projects:** Stay updated with team changes before merging.
2. **Monitoring Remote Branches:** Track remote branch progress without affecting local branches.
3. **Inspecting Changes:** Review updates before deciding to merge or rebase.

Basic Syntax and Usage :-

The basic syntax for git fetch is straightforward:

git fetch <remote> <branch>

- **<remote>**: The name of the remote repository (default is origin).
- **<branch> (optional)**: The specific branch you want to fetch

Example 1: Fetch All Branches from the Remote Repository

git fetch origin

This command fetches updates for all branches from the remote repository named origin.

Example 2: Fetch a Specific Branch

git fetch origin feature-branch

This command fetches only the updates from the feature-branch on the remote repository.

Difference Between Git Fetch and Git Pull :-

git fetch and git pull are often confused, but they serve different purposes:

Git Fetch	Git Pull
Checks for updates from remote without applying them	Fetches and merges changes into your current branch
Safe for reviewing before merging	May introduce merge conflicts immediately
Keeps local working directory unchanged	Alters working directory based on remote updates
Best for cautious syncing	Best when you're ready to integrate updates